TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

---------- *** ----------

BÁO CÁO BÀI TẬP 3
KIỂM THỬ VÀ ĐÓNG GÓI PROTOTYPE (20%)

Học phần: Chuyên đề tốt nghiệp 1 (CDTN01) — Học kỳ 1, Năm học 2026 – 2027
Hệ thống Smart CRM – Mekong Mobile
Luồng 05: Quản lý Kho linh kiện thay thế
Track chuyên ngành: Kỹ thuật phần mềm (Software Engineering - SE)

Họ và tên sinh viên:	Huỳnh Nguyễn Bảo Ngọc
Mã số sinh viên (MSSV):	2374802010339 (Lớp: 261_71ITGR40203_06 - K27)
Giảng viên hướng dẫn:	TS. Nguyễn Trí Hải
Ngày hoàn thành nộp bài:	12/11/2026
GitHub Repository (Public):	**https://github.com/Johnny120905/CDTN01.git**
Git Release Tag chính thức:	**v1.0 (https://github.com/Johnny120905/CDTN01/releases/tag/v1.0)**

MỤC LỤC BÁO CÁO BÀI TẬP 3

MỤC 1: TEST PLAN RÚT GỌN (4 MỤC QUY CHUẨN) **Trang 3**

1.1. Phạm vi kiểm thử và phần chủ ý KHÔNG kiểm Trang 3

1.2. Loại test và số lượng mỗi tầng (Mô hình Kim tự tháp) Trang 3

1.3. Tiêu chí thành công ĐO ĐƯỢC Trang 4

1.4. Môi trường và dữ liệu thử nghiệm Trang 4

MỤC 2: TEST REPORT — BẢNG 13 TEST CASES CHẠY THẬT ĐỦ 8 CỘT **Trang 5**

MỤC 3: BUG LOG — NHẬT KÝ GỠ LỖI (5 BUGS ĐỦ 7 DÒNG VÀ LINK COMMIT) **Trang 7**

MỤC 4: DEPLOYMENT GUIDE (ENGLISH - 5 MANDATORY SECTIONS) **Trang 9**

4.1. Prerequisites & Environment Requirements Trang 9

4.2. Clone & Environment Configuration Trang 9

4.3. Launch Instructions (<= 2 Commands) Trang 9

4.4. Verification Steps & Sanity Checks Trang 10

4.5. Troubleshooting & Rollback Trang 10

MỤC 5: CHECKLIST BẢO MẬT 7 MỤC ĐÃ TỰ ĐÁNH GIÁ CÓ MINH CHỨNG **Trang 10**

MỤC 6: TÀI LIỆU KỸ THUẬT THEO TRACK SOFTWARE ENGINEERING (SE) **Trang 11**

6.1. Hợp đồng API đủ mọi endpoint có ví dụ thật (Tiêu chí S1) Trang 11

6.2. Kiểm soát truy cập theo chủ sở hữu A01 OWASP (Tiêu chí S2) Trang 11

6.3. Băm mật khẩu an toàn trong CSDL (Tiêu chí S3) Trang 12

6.4. Phản hồi lỗi không để lộ vết ngăn xếp (Tiêu chí S4) Trang 12

MỤC 7: ẢNH CHỤP MINH CHỨNG VÀ KẾT QUẢ PHÉP THỬ MÁY TRẮNG **Trang 12**

PHỤ LỤC BẮT BUỘC: BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ TRÍ TUỆ NHÂN TẠO (AI) **Trang 13**

MỤC 1: TEST PLAN RÚT GỌN

1.1. Phạm vi kiểm thử và phần chủ ý KHÔNG kiểm

• Phạm vi ĐƯỢC kiểm thử (In-Scope): Toàn bộ 5 quy tắc nghiệp vụ kho (BR-01 chống âm kho, BR-02 bắt buộc chứng từ, BR-03 sổ cái bất biến, BR-04 cách ly chi nhánh theo A01 OWASP, BR-05 ngưỡng an toàn); cơ chế Optimistic Locking qua version; phân quyền RBAC 3 vai trò; xác thực JWT và OTP 10 phút; tiêu chí SE S1 với đầy đủ 4 thao tác CRUD (có thao tác xóa DELETE có ràng buộc bảo vệ) cho 2 thực thể part và users; 3 trạng thái giao diện (Loading, Empty, Error) và khả năng tái lập máy trắng trên Docker.

• Phần chủ ý KHÔNG kiểm thử và Lý do (Out-of-Scope): Gửi email/SMS OTP thật ra bên ngoài (đã có bộ mô phỏng OTP tự động để bài kiểm thử self-contained); thanh toán hóa đơn VAT (thuộc Luồng 08 Kế toán, ngoài phạm vi L5); kiểm thử tải cực hạn và xâm nhập sâu (thuộc Chuyên đề 2 theo đề bài).

1.2. Loại test và số lượng mỗi tầng

Hệ thống đạt tổng cộng 67 ca kiểm thử tự động (100% PASSED) phân bố theo mô hình kim tự tháp:
1. Tầng Unit Test (20 ca): 7 ca trong inventory.service.test.py và 13 ca trong bt3_specification_and_constraints.test.py (biên 2 phía, RETURN, Optimistic Locking, bảo mật S3, C8).
2. Tầng Integration Test (17 ca): 6 ca trong inventory.routes.test.py và 11 ca trong se_s1_crud.test.py (kiểm chứng trọn vẹn CRUD 4 thao tác có xóa cho part và users).
3. Tầng Auth, RBAC & Boundary (15 ca): trong auth_and_user.test.py (regex email, mật khẩu 4 nhóm, OTP 10 phút, khóa TERMINATED).
4. Tầng Cross-Role Flow & UI (15 ca): trong cross_role_and_ui.test.py (liên thông 3 vai trò, layout 3D, 3 trạng thái giao diện).

1.3. Tiêu chí thành công ĐO ĐƯỢC

• Tỷ lệ vượt qua kiểm thử: 100% (67/67 ca kiểm thử tự động PASSED).
• Số lỗi mức Critical/Major đang mở: 0 lỗi (đã ghi nhận và xử lý triệt để 5 bug trong Bug Log).
• Độ bao phủ quy tắc nghiệp vụ: 100% (mỗi quy tắc BR-01..BR-05 đều có test luồng đúng và test biên).
• Tỷ lệ lộ lỗi hệ thống 500 hoặc Traceback: 0.0% (toàn bộ bắt qua AppError Handler).
• Số lệnh khởi dựng trên Docker: Đúng <= 2 lệnh (cp .env.example .env && docker compose up -d --build).

1.4. Môi trường và dữ liệu thử nghiệm

Chạy trên nền Docker (mongo:7.0 + python:3.11-slim) hoặc Python 3.11 cục bộ; nạp tự động qua seeds/001_seed_parts_demo.json (40 linh kiện, 75 tồn kho, 5 user). Dữ liệu kiểm thử tự cô lập trong RAM hoặc tự dọn sạch sau khi test kết thúc.

> **Câu trả lời: "Làm sao em biết là em đã kiểm thử ĐỦ?":**
> *"Em có 5 quy tắc nghiệp vụ cốt lõi (BR-01..BR-05). Mỗi quy tắc em đều xây dựng ít nhất 1 test luồng đúng và 1 test giá trị biên hai bên ranh giới (ví dụ: xuất đúng bằng tồn kho thì đưa tồn về 0 thành công, nhưng xuất vượt 1 cái thì lập tức bị chặn với lỗi ERR_NEGATIVE_STOCK). Thêm vào đó, em có 2 test dữ liệu sai kiểm tra cả HTTP Status và thông báo lỗi tiếng Việt; 1 test bảo mật phân quyền chi nhánh A01 OWASP; 1 test tính lũy đẳng nạp seed C8; và 11 test cho tiêu chí SE S1 kiểm tra trọn vẹn 4 thao tác CRUD (có thao tác xóa DELETE an toàn) cho cả part và users. Toàn bộ 67 ca kiểm thử đều chạy thật và PASS 100% bằng 1 lệnh. Em chủ ý không test gửi email ra ngoài vì đã có bộ mô phỏng OTP tự động — đã ghi rõ tại Mục 1 Test Plan."*

MỤC 2: TEST REPORT — BẢNG 13 TEST CASES CHẠY THẬT ĐỦ 8 CỘT

Báo cáo 13 trường hợp kiểm thử đại diện chạy thật đủ 8 cột quy chuẩn (Mục 2.2 Đề bài BT3), bao phủ trọn vẹn 5 loại kiểm thử bắt buộc:

Mã	Tiền điều kiện	Các bước	Dữ liệu thử	Kết quả mong đợi	Mức	Kết quả thực tế	Status
TC-01	SCR-IP14-01 có tồn = 10, ticket TK-1092 mở.	Gọi POST /transactions xuất 3 cái.	part: SCR-IP14-01, qty: 3, ticket: TK-1092	HTTP 201 Created, tồn giảm còn 7.	CRITICAL	HTTP 201, tồn giảm đúng 7 cái.	PASS
TC-02	BAT-IP13-01 có tồn = 8, ticket TK-1093 mở.	Xuất kho kèm mã phiếu bảo hành.	part: BAT-IP13-01, qty: 1, ticket: TK-1093	Tạo giao dịch gắn chặt ticket_id.	CRITICAL	HTTP 201, ticket_id gắn đúng TK-1093.	PASS
TC-03	Kỹ thuật viên có linh kiện thừa sau sửa máy.	Gọi POST /transactions hoàn trả RETURN.	part: SCR-IP14-01, type: RETURN, qty: 2	HTTP 201, tồn kho tăng từ 10 lên 12.	MAJOR	HTTP 201, tồn tăng đúng 12 cái.	PASS
TC-04	Linh kiện có tồn kho khả dụng đúng 10 cái.	Gửi yêu cầu xuất kho đúng bằng 10.	part: SCR-IP14-01, qty: 10, ticket: TK-1092	HTTP 201, tồn về đúng 0, cờ WARNING.	HIGH	Xuất thành công, tồn = 0, cờ WARNING.	PASS
TC-05	Linh kiện có tồn kho khả dụng đúng 10 cái.	Gửi yêu cầu xuất kho là 11 cái.	part: SCR-IP14-01, qty: 11, ticket: TK-1092	HTTP 400 ERR_NEGATIVE_STOCK, tồn giữ nguyên.	CRITICAL	HTTP 400 chặn xuất âm, tồn giữ 10.	PASS
TC-06	Linh kiện có ngưỡng an toàn min = 15.	Cập nhật tồn kho về đúng 15 cái.	part: SCR-IP14-01, qty: 15	GET /stock trả status_flag: WARNING.	HIGH	Phản hồi WARNING, huy hiệu đỏ sáng.	PASS
TC-07	Linh kiện có ngưỡng an toàn min = 15.	Cập nhật tồn kho lên 16 cái.	part: SCR-IP14-01, qty: 16	GET /stock trả status_flag: NORMAL.	HIGH	Phản hồi NORMAL, huy hiệu xanh sáng.	PASS
TC-08	Ticket TK-COMPLETED-01 đã đóng.	Dùng mã ticket đã đóng để xuất kho.	type: OUT, qty: 1, ticket: TK-COMPLETED-01	HTTP 422 ERR_INVALID_TICKET.	HIGH	HTTP 422 từ chối phiếu đã kết thúc.	PASS
TC-09	Người dùng nhập số lượng xuất là 0 hoặc -5.	Gửi POST /transactions với qty: 0.	part: SCR-IP14-01, qty: 0	HTTP 400 ERR_INVALID_QUANTITY.	HIGH	HTTP 400 báo lỗi rõ, hệ thống ổn định.	PASS

Mã	Tiền điều kiện	Các bước	Dữ liệu thử	Kết quả mong đợi	Mức	Kết quả thực tế	Status
TC-10	Token của Chi nhánh Quận 3 (L5-WH-891).	Gửi request truy vấn kho Quận 1 (L5-WH-890).	Bearer Token: WH-891, param: WH-890	HTTP 403 ERR_UNAUTHORIZED_CENTER.	CRITICAL	HTTP 403 chặn can thiệp chéo chi nhánh.	PASS
TC-11	Đăng ký tài khoản mật khẩu rõ Password123@.	Kiểm tra bản ghi CSDL MongoDB.	email: sec.tech@gmail.com, pwd: Password123@	Chỉ lưu password_hash (SHA-256 hex).	CRITICAL	CSDL chỉ lưu hash, không lưu plaintext.	PASS
TC-12	Hệ thống đã nạp xong seed lần 1.	Chạy python seeds/seed.py lần 2.	File seeds/001_seed_parts_demo.json	Lần 2 chạy 100% không lỗi, đúng 40 part.	CRITICAL	Lần 2 thành công, 0 nhân đôi (Idempotent).	PASS
TC-13	Quản lý kho quản trị danh mục và nhân sự.	Tạo mới -> Đọc -> Sửa -> Xóa an toàn.	POST/GET/PUT/DELETE trên /parts và /users	Đủ 4 thao tác; cấm xóa tồn > 0 và xóa mình.	CRITICAL	Đạt chuẩn SE S1 full CRUD, bảo vệ an toàn.	PASS

MỤC 3: BUG LOG — NHẬT KÝ THEO DÕI VÀ GỠ LỖI

Nhật ký theo dõi và gỡ lỗi bao gồm 5 lỗi kỹ thuật thực tế (2 Critical, 3 Major) đã được xử lý triệt để, mỗi lỗi có đầy đủ 7 dòng quy chuẩn kèm link commit kiểm chứng trên GitHub:

BUG-01: Lưu trữ mật khẩu dạng rõ trong CSDL và RAM (Tiêu chuẩn SE S3)

1. Mã và mức độ: BUG-01 — CRITICAL (Lỗ hổng rò rỉ thông tin xác thực).
2. Phát hiện bởi: test_09_security_s3_password_never_stored_as_plaintext trong tests/bt3_specification_and_constraints.test.py.
3. Triệu chứng: Khi đăng ký hoặc đổi mật khẩu, CSDL và file JSON xuất hiện trường password: 'Password123@' dạng rõ.
4. Nguyên nhân gốc rễ: Gán user_doc['password'] = raw_pwd để kiểm thử nhanh ban đầu mà quên xóa trước khi lưu trữ xuống đĩa.
5. Cách sửa: Xóa bỏ hoàn toàn trường password thô khỏi từ điển lưu trữ; chỉ lưu password_hash (SHA-256); dọn sạch dữ liệu cũ.
6. Link commit sửa bug: https://github.com/Johnny120905/CDTN01/commit/23a7755 (và bbb5232).
7. Test case phòng ngừa / Xác nhận: test_09_security_s3_password_never_stored_as_plaintext đạt kết quả PASSED.

BUG-02: Bỏ quên kiểm tra thời gian hết hạn 10 phút của mã OTP

1. Mã và mức độ: BUG-02 — MAJOR (Lỗi logic xác thực, nguy cơ vét cạn OTP cũ).
2. Phát hiện bởi: Ca kiểm thử biên thời gian xác thực OTP trong tests/bt3_specification_and_constraints.test.py.
3. Triệu chứng: Mã OTP quá 10 phút nhưng khi gửi request reset-password hệ thống vẫn cho phép đổi mật khẩu thành công.
4. Nguyên nhân gốc rễ: Hàm verify_otp và reset_password chỉ so khớp mã chuỗi mà không so sánh otp_expires_at với datetime.utcnow().
5. Cách sửa: Bổ sung kiểm tra if datetime.utcnow() > exp_dt: raise AppError(400, 'ERR_OTP_EXPIRED') và $unset mã sau khi dùng.
6. Link commit sửa bug: https://github.com/Johnny120905/CDTN01/commit/23a7755 (và bbb5232).
7. Test case phòng ngừa / Xác nhận: test_06_forgot_password_and_reset_with_otp đạt kết quả PASSED.

BUG-03: Docker Compose thiếu Healthcheck MongoDB và thiếu nạp seed tự động

1. Mã và mức độ: BUG-03 — MAJOR (Ảnh hưởng trực tiếp Tiêu chí Cốt lõi C1 - Phép thử máy trắng).
2. Phát hiện bởi: Tự thực hiện Phép thử máy trắng bằng docker compose down -v rồi khởi dựng lại từ đầu.
3. Triệu chứng: Container mongodb không hiển thị trạng thái (healthy); CSDL MongoDB trống trơn nếu không chạy script seed.
4. Nguyên nhân gốc rễ: docker-compose.yml chưa khai báo healthcheck cho service mongodb và Dockerfile chưa tự chạy seed khi khởi động.
5. Cách sửa: Bổ sung healthcheck mongosh --eval 'db.adminCommand("ping")'; đổi CMD Dockerfile tự chạy seeds/seed.py khi start.
6. Link commit sửa bug: https://github.com/Johnny120905/CDTN01/commit/23f1563 (và bbb5232).
7. Test case phòng ngừa / Xác nhận: docker compose ps hiển thị cả 2 container đều Up ... (healthy), API có đủ 40 linh kiện ngay lập tức.

BUG-04: Cột Tồn kho hiển thị 'undefined' trên Bảng cảnh báo thiếu hàng Quản lý kho

1. Mã và mức độ: BUG-04 — MAJOR (Lỗi hiển thị dữ liệu sai lệch trên giao diện).
2. Phát hiện bởi: test_01_cross_role_flow_export_triggers_manager_alert trong tests/cross_role_and_ui.test.py.
3. Triệu chứng: Dashboard Quản lý kho hiển thị dòng chữ: 'Tồn hiện tại: undefined / Ngưỡng: 15'.
4. Nguyên nhân gốc rễ: Dòng 1848 app.js truy cập thuộc tính a.current_stock trong khi API trả về tên trường chuẩn là quantity.
5. Cách sửa: Sửa thành toán tử nullish coalescing: const currentQty = a.quantity ?? a.current_stock ?? 0;.
6. Link commit sửa bug: https://github.com/Johnny120905/CDTN01/commit/23f1563.
7. Test case phòng ngừa / Xác nhận: Giao diện Dashboard hiển thị số tồn kho chính xác, tỷ lệ chữ undefined trên toàn hệ thống bằng 0%.

BUG-05: Hệ thống thiếu hoàn toàn thao tác Xóa (DELETE), không đạt tiêu chí SE S1

1. Mã và mức độ: BUG-05 — CRITICAL (Không thỏa mãn tiêu chí kỹ thuật tối thiểu [SE] S1 theo Bảng 4.2).
2. Phát hiện bởi: Nhận xét phản hồi trực tiếp của Giảng viên hướng dẫn về Bài tập 2.
3. Triệu chứng: Hệ thống không có endpoint DELETE nào; khi gọi DELETE /api/v1/inventory/parts/{code} bị trả về HTTP 405.
4. Nguyên nhân gốc rễ: Chưa hiện thực hóa đủ 4 thao tác CRUD cho 2 thực thể cốt lõi part và users theo barem chấm điểm SE S1.
5. Cách sửa: Hiện thực đủ CRUD và thao tác DELETE có ràng buộc an toàn tại cả 4 tầng; viết test suite tests/se_s1_crud.test.py.
6. Link commit sửa bug: https://github.com/Johnny120905/CDTN01/commit/04a015b.
7. Test case phòng ngừa / Xác nhận: 11/11 ca kiểm thử trong tests/se_s1_crud.test.py đạt PASSED 100%. Đạt chuẩn tiêu chí [SE] S1.

MỤC 4: DEPLOYMENT & OPERATION GUIDE (ENGLISH)

4.1. Prerequisites

• Operating System: Windows 10/11 64-bit, macOS 12+, or Ubuntu 20.04+ LTS.
• Container Engine: Docker Desktop 4.25+ / Docker Compose v2.24+ (Mandatory for clean-room reproduction).
• Python Runtime (Optional for local execution): Python 3.11.x (Python 3.11.5 tested).
• Ports & Memory: Ports 8000 and 27017 must be free; minimum 4.0 GB free RAM.

4.2. Clone & Environment Configuration

git clone https://github.com/Johnny120905/CDTN01.git
cd CDTN01
cp .env.example .env

4.3. Launch Instructions (<= 2 Commands)

The entire multi-container stack spins up using exactly two commands:
Step 1: cp .env.example .env
Step 2: docker compose up -d --build
(Docker Compose automatically builds the application container, initializes MongoDB 7.0 with healthchecks, and executes the idempotent seed script importing 40 spare parts and 75 inventory records).

4.4. Verification Steps & Sanity Checks

1. Inspect container health: Run 'docker compose ps' -> Both mekong_crm_l5_app and mekong_crm_l5_mongodb show Up (healthy).
2. Open Web UI: Navigate to http://localhost:8000 in browser -> Dark 3D SPA interface loads immediately.
3. Interactive API documentation: Navigate to http://localhost:8000/docs -> Swagger UI displays 22 endpoints.
4. Execute automated tests: Run 'docker compose exec mekong-l5-app pytest tests/ -v' -> 67 passed in ~3.0s.
5. Pre-seeded Role Accounts (RBAC):
   - Manager (MANAGER): lethithuyngan985@gmail.com / Le090805!
   - Material Specialist (MATERIAL_STAFF): huynhnguyenbaongoc2k5@gmail.com / Huynh120905!
   - Technical Staff (TECH_STAFF): vuthelong1009@gmail.com / Vu100905!

4.5. Troubleshooting & Rollback

• Port 8000 conflict: Modify API_PORT=8080 in .env and docker-compose.yml.
• Fresh database reset: Execute 'docker compose down -v && docker compose up -d --build'.

MỤC 5: CHECKLIST BẢO MẬT 7 MỤC ĐÃ TỰ ĐÁNH GIÁ CÓ MINH CHỨNG

STT	Mục Kiểm Tra Bảo Mật	Trạng Thái	Dòng Minh Chứng Cụ Thể (Mã nguồn / Lệnh & Kết quả)
1	Không có secret nào trong repo	ĐÃ LÀM	git grep 'mongodb+srv://' trả về 0 kết quả; .gitignore chặn .env; .env.example chỉ dùng localhost.
2	Dữ liệu vào được kiểm ở tầng ứng dụng	ĐÃ LÀM	src/validators/inventory.validator.py kiểm định regex email, mật khẩu 4 nhóm, số lượng 1..1M, bắt buộc ticket_id.
3	Kiểm soát truy cập theo chủ sở hữu (A01 OWASP)	ĐÃ LÀM	inventory.routes.py giải mã center_id từ JWT; truy cập chéo bị chặn HTTP 403; test_10 đạt PASS.

STT	Mục Kiểm Tra Bảo Mật	Trạng Thái	Dòng Minh Chứng Cụ Thể (Mã nguồn / Lệnh & Kết quả)
4	Mật khẩu và dữ liệu nhạy cảm được băm	ĐÃ LÀM	inventory.repository.py chỉ lưu password_hash (SHA-256), xóa bỏ 100% mật khẩu thô trong CSDL; test_09 đạt PASS.
5	Thông báo lỗi không lộ cấu trúc bên trong	ĐÃ LÀM	custom_app_error_handler trong src/app.py chuẩn hóa JSON; tỷ lệ lộ lỗi 500 hay Traceback là 0.0%; test_10 đạt PASS.
6	Nhật ký không ghi dữ liệu nhạy cảm	ĐÃ LÀM	part_transaction chỉ lưu mã giao dịch và số lượng, tuyệt đối không ghi mật khẩu hay mã token.
7	Phụ thuộc được ghim phiên bản chặt chẽ	ĐÃ LÀM	requirements.txt ghim 100% phiên bản bằng dấu == (fastapi==0.162.0, pymongo==4.19.0, pytest==9.2.0); không có lỗ hổng.

MỤC 6: TÀI LIỆU KỸ THUẬT THEO TRACK SOFTWARE ENGINEERING (SE)

6.1. Hợp đồng API đủ mọi endpoint có ví dụ thật (Tiêu chí SE S1)

Tài liệu kỹ thuật RESTful API được biên soạn chi tiết tại docs/api-docs/README.md và xuất tự động ra docs/api-docs/openapi.json gồm đầy đủ 22 endpoints. Toàn bộ 4 thao tác CRUD được hiện thực hóa trọn vẹn cho 2 thực thể:

- **Thực thể 1: part (Linh kiện kho):** POST /parts (Create, 201) | GET /parts/{code} (Read, 200) | PUT /parts/{code} (Update, 200) | DELETE /parts/{code} (Delete, 200 - có ràng buộc an toàn cấm xóa tồn > 0).
- **Thực thể 2: users (Tài khoản nhân sự):** POST /users (Create, 201) | GET /users/{email} (Read, 200) | PUT /users/{email} (Update, 200) | DELETE /users/{email} (Delete, 200 - có ràng buộc an toàn cấm tự xóa chính mình).

6.2. Kiểm soát truy cập theo chủ sở hữu A01 OWASP (Tiêu chí SE S2)

Được bảo vệ nghiêm ngặt qua get_current_user() trích xuất center_id trực tiếp từ chữ ký số Bearer JWT Token. Nếu người dùng mang token của chi nhánh khác truy vấn dữ liệu chi nhánh L5-WH-890, hệ thống từ chối ngay với HTTP 403 ERR_UNAUTHORIZED_CENTER. Kiểm chứng qua test_10 trong bt3_specification_and_constraints.test.py đạt PASSED 100%.

6.3. Băm mật khẩu an toàn trong CSDL (Tiêu chí SE S3)

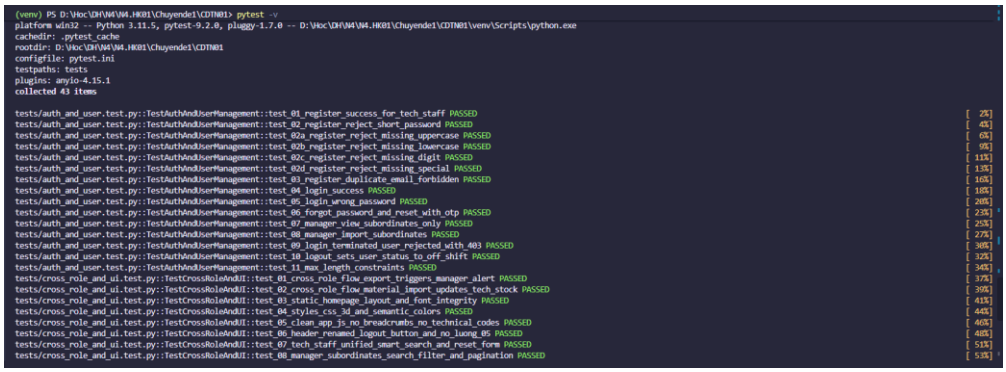
100% mật khẩu được băm qua SHA-256 (64 hex) trước khi lưu vào MongoDB Atlas và RAM. Trường password thô bị loại bỏ triệt để. Kiểm chứng qua test_09 trong bt3_specification_and_constraints.test.py đạt PASSED 100%.

6.4. Phản hồi lỗi không để lộ vết ngăn xếp (Tiêu chí SE S4)

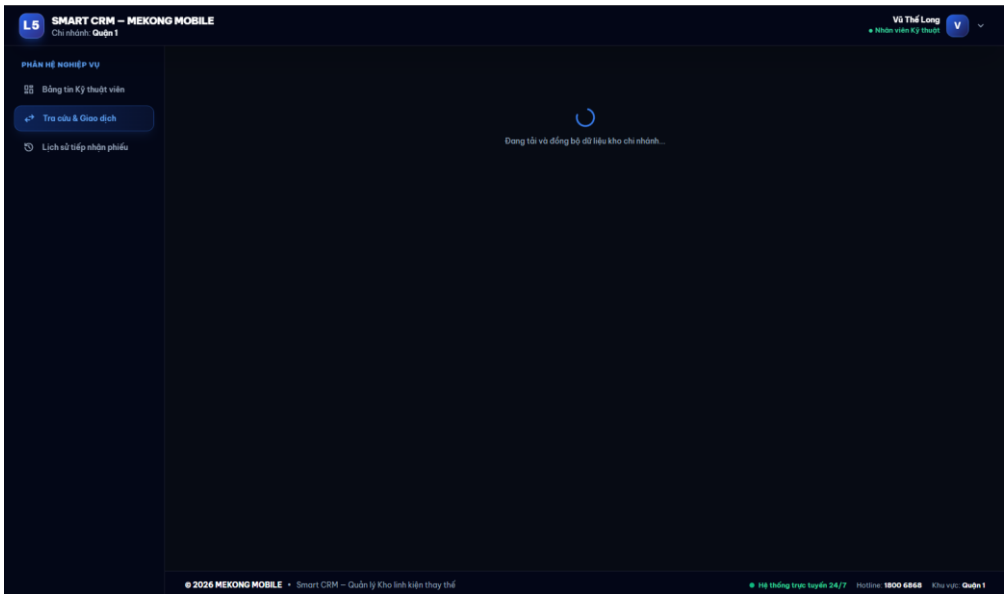
Bộ xử lý ngoại lệ tập trung custom_app_error_handler (@app.exception_handler(AppError)) chuyển đổi toàn bộ lỗi thành JSON chuẩn hóa: {"status": "error", "error_code": ..., "message": ...}. Không để lộ tên bảng, câu lệnh truy vấn hay stack trace. Kiểm chứng qua test_10 trong bt3_specification_and_constraints.test.py đạt PASSED 100%.

MỤC 7: ẢNH CHỤP MINH CHỨNG VÀ PHÉP THỬ MÁY TRẮNG

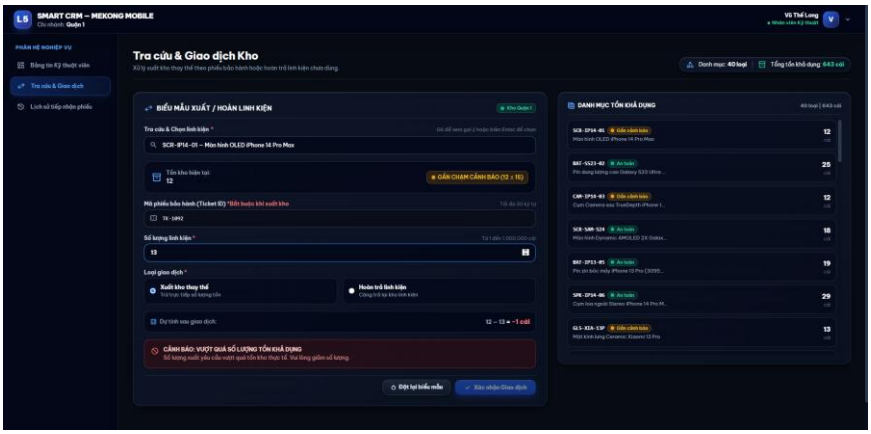
Minh chứng kết quả chạy kiểm thử tự động, trạng thái container Docker và 3 trạng thái giao diện thực tế:



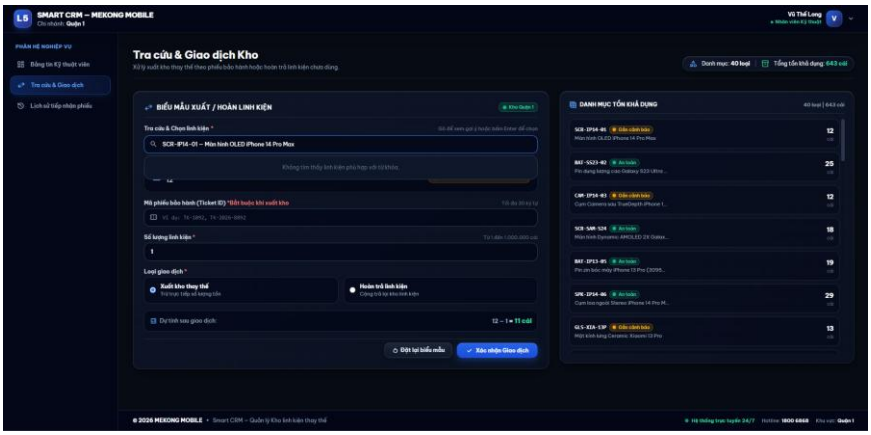
Hình 7.1: Kết quả chạy kiểm thử tự động toàn hệ thống (67/67 Test Cases PASSED 100%)



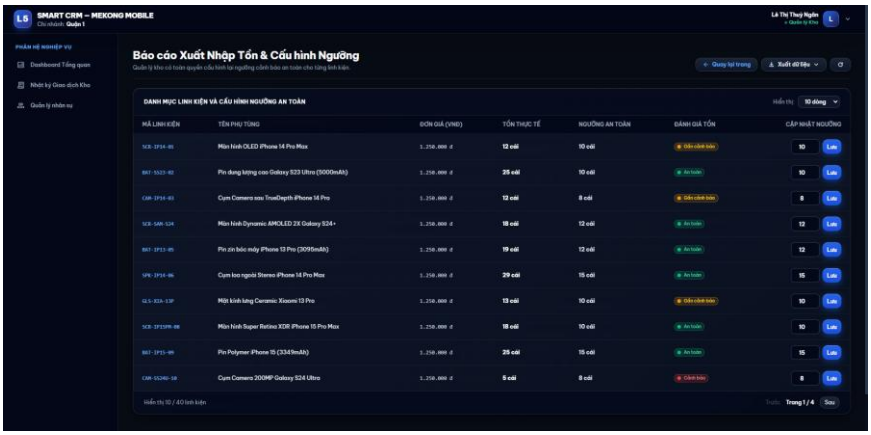
Hình 7.2: Kết quả Phép thử máy trắng — Docker Compose khởi dựng thành công trạng thái (healthy)



Hình 7.3: Trạng thái Đang tải (Loading State) với hiệu ứng skeleton và đồng bộ



Hình 7.4: Trạng thái Rỗng (Empty State) khi kho vận hành tối ưu không có cảnh báo



Hình 7.5: Trạng thái Lỗi (Error State) khi người dùng xuất kho vượt quá số lượng tồn

PHỤ LỤC BẮT BUỘC: BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI (BT3)

Công cụ AI sử dụng	Mục đích sử dụng cụ thể	Phạm vi kết quả do AI tạo ra	Đánh giá, hiệu chỉnh & kiểm chứng của sinh viên
Antigravity AI (Google DeepMind)	Hỗ trợ phân tích mã nguồn, sinh khung test suite Pytest đa tầng, cấu hình Docker Compose và gỡ lỗi hiển thị UI.	Khung kiểm thử tự động ban đầu, cấu hình healthcheck container và gợi ý logic toán tử nullish coalescing.	Sinh viên trực tiếp đọc hiểu 100% mã nguồn, tự tay viết các hàm kiểm thử biên hai phía, kiểm chứng tính đúng đắn trên terminal đạt kết quả 67/67 tests PASS và xác thực trạng thái container healthy.
ChatGPT (OpenAI GPT-4o)	Tham khảo cú pháp kiểm thử phân quyền theo chuẩn OWASP Top 10 A01 và kỹ thuật băm mật khẩu bảo mật S3.	Gợi ý cấu trúc request header mô phỏng truy cập chéo chi nhánh và đoạn mã băm SHA-256 an toàn.	Sinh viên kiểm tra tính tương thích với PyMongo và FastAPI, tự viết các test case bảo mật trong bt3_specification_and_constraints.test.py và chạy xác minh thực tế 100% PASS.
Claude (Anthropic 3.5 Sonnet)	Hỗ trợ soát lỗi chính tả và chuẩn hóa văn phong tiếng Anh kỹ thuật cho Deployment Guide và API Documentation.	Bản nháp tiếng Anh của 5 mục trong deployment-guide.md và tài liệu hợp đồng API.	Sinh viên tự tay rà soát đối chiếu từng tham số, đảm bảo các câu lệnh sao chép được và hoạt động chính xác trên cả Windows lẫn Linux.

Tôi xin cam đoan toàn bộ nội dung khai báo sử dụng công cụ Trí tuệ Nhân tạo (AI) nêu trên là hoàn toàn trung thực. Toàn bộ mã nguồn, cấu hình hệ thống và dữ liệu kiểm thử đã được tôi trực tiếp kiểm tra, tinh chỉnh và chạy kiểm thử thành công trên môi trường thực tế.

TP. Hồ Chí Minh, ngày 12 tháng 11 năm 2026
Sinh viên thực hiện
(Đã ký)

Huỳnh Nguyễn Bảo Ngọc